Python Libraries

NumPy | Pandas | Matplotlib | Seaborn

Quick notes + interview questions and answers for Data Analyst roles. Every answer is short enough to say out loud, with code you can write on a whiteboard.

Part	Focus	Questions
1. NumPy	Arrays, broadcasting, vectorization	10
2. Pandas	Cleaning, groupby, merge, time series	18
3. Matplotlib	Figures, axes, chart choice	7
4. Seaborn	Statistical plots, themes	6
5. Scenarios	Rapid-fire + project questions	5

Part 1 - NumPy

NumPy gives a fast n-dimensional array (ndarray) and vectorized math. It is the base that Pandas, Matplotlib and Seaborn are built on.

Quick Notes

- ndarray stores one data type in a contiguous block of memory, so it is faster and smaller than a Python list.
- Vectorization = whole-array operations without Python loops (runs in C).
- Key attributes: shape, ndim, size, dtype. Axis 0 = rows (down), axis 1 = columns (across).
- Missing values are np.nan (a float). Use np.isnan, np.nanmean, np.nansum.

Q1. Why is a NumPy array faster than a Python list?

Answer: A NumPy array holds same-type values in one contiguous memory block and runs operations in compiled C code (vectorization). A list stores pointers to separate Python objects, so every operation has interpreter overhead.

```
import numpy as np
a = np.arange(1_000_000)
b = a * 2                 # vectorized, no loop
c = [x * 2 for x in range(1_000_000)]   # much slower
```

Q2. What is broadcasting?

Answer: Broadcasting lets NumPy do arithmetic on arrays of different shapes by stretching the smaller one. Shapes are compared from the right; each pair must be equal or one of them must be 1.

```
a = np.array([[1], [2], [3]])      # (3,1)
b = np.array([10, 20, 30])         # (3,)
print(a + b)                        # result shape (3,3)
```

Q3. Difference between a view and a copy?

Answer: Slicing returns a view (shares memory, so changes affect the original). Fancy or boolean indexing returns a copy. Use .copy() when you want an independent array.

```
a = np.arange(5); v = a[1:3]; v[0] = 99   # a changes too
c = a[[1, 2]].copy()                       # independent
```

Q4. reshape vs flatten vs ravel?

Answer: reshape changes the shape without changing data. flatten always returns a 1-D copy. ravel returns a 1-D view when possible (faster).

```
a = np.arange(6).reshape(2, 3)
a.flatten()   # copy
a.ravel()     # view if possible
a.reshape(-1) # -1 = infer the size
```

Q5. np.where and boolean masking - how do you use them?

Answer: A boolean mask filters elements by a condition. np.where(cond, x, y) picks x where the condition is True, else y (a vectorized if-else).

```
a = np.array([5, 12, 8, 20])
a[a > 10]                          # [12 20]
np.where(a > 10, 'high', 'low')
```

Q6. What does axis mean in sum, mean and other aggregations?

Answer: axis=0 collapses rows and gives one result per column; axis=1 collapses columns and gives one result per row. axis=None uses the whole array.

```
m = np.array([[1, 2], [3, 4]])
m.sum(axis=0)   # [4 6]
m.sum(axis=1)   # [3 7]
```

Q7. np.array vs np.arange vs np.linspace?

Answer: np.array builds from existing data. np.arange(start, stop, step) uses a step size (stop excluded). np.linspace(start, stop, n) gives n evenly spaced points (stop included).

```
np.arange(0, 10, 2)    # [0 2 4 6 8]
np.linspace(0, 1, 5)   # [0. 0.25 0.5 0.75 1.]
```

Q8. How do you handle NaN values in NumPy?

Answer: NaN propagates through normal functions, so use the nan-aware versions: np.nanmean, np.nansum, np.nanmax. Detect with np.isnan and replace with np.nan_to_num or masking.

```
a = np.array([1, np.nan, 3])
a.mean()           # nan
np.nanmean(a)      # 2.0
a[~np.isnan(a)]    # drop NaN
```

Q9. How do you stack or join arrays?

Answer: np.concatenate joins along an existing axis; np.vstack stacks vertically (rows); np.hstack stacks horizontally (columns); np.stack creates a new axis.

```
a = np.array([1, 2]); b = np.array([3, 4])
np.vstack([a, b])   # [[1 2],[3 4]]
np.hstack([a, b])   # [1 2 3 4]
```

Q10. What are np.mean, np.median and np.std used for in analysis?

Answer: Mean is the average and is sensitive to outliers; median is the middle value and is robust to outliers; std measures spread. Use ddof=1 for the sample standard deviation.

```
data = np.array([10, 12, 11, 100])
np.mean(data), np.median(data), np.std(data, ddof=1)
```

Part 2 - Pandas

Pandas is the main tool for tabular data. Series = one column (1-D labeled array). DataFrame = a table of Series sharing one index.

Quick Notes

- First look at any dataset: df.head(), df.info(), df.describe(), df.shape, df.isna().sum(), df.duplicated().sum().
- Cleaning order: fix dtypes -> handle missing -> drop duplicates -> fix text -> filter outliers -> feature columns.
- loc = label based (end inclusive). iloc = position based (end exclusive).
- Never loop over rows when a vectorized method exists. Use apply only as a last resort.

Q11. Series vs DataFrame?

Answer: A Series is a single labeled column (1-D). A DataFrame is a 2-D table made of several Series that share the same index. Each DataFrame column is a Series.

```
import pandas as pd
s = pd.Series([10, 20, 30], name='sales')
df = pd.DataFrame({'city': ['Delhi', 'Noida'], 'sales': [100, 80]})
```

Q12. loc vs iloc?

Answer: loc selects by label (and the end label is included). iloc selects by integer position (end excluded). With a default RangeIndex they look similar but behave differently when sliced.

```
df.loc[0:2, 'city']     # rows labeled 0,1,2
df.iloc[0:2, 0]         # positions 0,1 only
```

Q13. How do you handle missing values?

Answer: First inspect with isna().sum(). Then drop (dropna) when few rows are affected, or fill (fillna) with mean/median/mode or a constant; use ffill/bfill for time series. The choice depends on how much is missing and why.

```
df.isna().sum()
df['age'] = df['age'].fillna(df['age'].median())
df = df.dropna(subset=['customer_id'])
```

Q14. How do you remove duplicates?

Answer: Use duplicated() to inspect and drop_duplicates() to remove. subset limits the check to certain columns and keep chooses which copy to keep.

```
df.duplicated().sum()
df = df.drop_duplicates(subset=['order_id'], keep='first')
```

Q15. groupby - how does it work?

Answer: groupby follows split-apply-combine: split rows into groups, apply an aggregation to each group, combine into one result. Use agg for several metrics at once.

```
df.groupby('city')['sales'].sum()
df.groupby('city').agg(total=('sales', 'sum'), avg=('sales', 'mean'), orders=('order_id', 'count'))
```

Q16. merge vs join vs concat?

Answer: merge is the SQL-style join on columns (how = inner, left, right, outer). join merges on the index. concat stacks DataFrames along rows (axis=0) or columns (axis=1) without matching keys.

```
pd.merge(orders, customers, on='customer_id', how='left')
pd.concat([jan, feb], ignore_index=True)
```

Q17. How do you check and change data types?

Answer: Check with df.dtypes or df.info(). Convert with astype, pd.to_numeric(errors='coerce') for messy numbers, pd.to_datetime for dates, and category dtype to save memory on repeated text.

```
df['price'] = pd.to_numeric(df['price'], errors='coerce')
df['date'] = pd.to_datetime(df['date'])
df['city'] = df['city'].astype('category')
```

Q18. apply vs map vs applymap?

Answer: map works element-wise on a Series. apply works on a Series, or on rows/columns of a DataFrame (axis). applymap (renamed DataFrame.map in newer versions) works element-wise on the whole DataFrame. Prefer vectorized operations over all three for speed.

```
df['name'] = df['name'].map(str.title)
df['total'] = df.apply(lambda r: r['qty'] * r['price'], axis=1)   # slow
df['total'] = df['qty'] * df['price']                              # fast
```

Q19. pivot_table vs groupby?

Answer: Both aggregate. groupby returns a long result; pivot_table reshapes it into a spreadsheet-like grid with rows, columns and a values field, like an Excel pivot.

```
pd.pivot_table(df, index='city', columns='month', values='sales', aggfunc='sum', fill_value=0)
```

Q20. How do you detect outliers?

Answer: Use the IQR rule (below Q1 - 1.5*IQR or above Q3 + 1.5*IQR), z-scores, or a boxplot. Then decide: remove, cap (winsorize), or keep if it is a real extreme value.

```
q1, q3 = df['sales'].quantile([.25, .75])
iqr = q3 - q1
clean = df[df['sales'].between(q1 - 1.5*iqr, q3 + 1.5*iqr)]
```

Q21. How do you work with dates?

Answer: Convert with pd.to_datetime, then use the .dt accessor to extract parts. Set the date as index and use resample for time-based aggregation.

```
df['date'] = pd.to_datetime(df['date'])
df['month'] = df['date'].dt.month_name()
df.set_index('date').resample('M')['sales'].sum()
```

Q22. What is the SettingWithCopyWarning?

Answer: It appears when you assign to a slice that might be a copy of another DataFrame, so the change may not reach the original. Avoid it by using .loc for assignment or by calling .copy() when you create the subset.

```
sub = df[df['sales'] > 50].copy()
sub['flag'] = 1
df.loc[df['sales'] > 50, 'flag'] = 1
```

Q23. How do you rename columns, sort and filter?

Answer: rename takes a mapping; sort_values sorts by columns; filter with boolean conditions combined with & and | (each condition in brackets), or with query for readable code.

```
df = df.rename(columns={'amt': 'amount'})
df.sort_values('amount', ascending=False)
df[(df['amount'] > 100) & (df['city'] == 'Delhi')]
```

Q24. value_counts, nunique and unique?

Answer: value_counts gives the frequency of each value, nunique counts distinct values, unique returns the distinct values themselves.

```
df['city'].value_counts(normalize=True)
df['city'].nunique()
df['city'].unique()
```

Q25. How do you clean text columns?

Answer: Use the .str accessor: strip, lower, replace, contains, split. Standardising case and whitespace before groupby or merge prevents silent mismatches.

```
df['city'] = df['city'].str.strip().str.title()
df['email_ok'] = df['email'].str.contains('@', na=False)
```

Q26. How do you handle a large CSV that does not fit in memory?

Answer: Read in chunks with chunksize, load only needed columns with usecols, give columns smaller dtypes, and aggregate each chunk before combining.

```
parts = []
for chunk in pd.read_csv('big.csv', chunksize=100_000, usecols=['city', 'sales']):
    parts.append(chunk.groupby('city')['sales'].sum())
result = pd.concat(parts).groupby(level=0).sum()
```

Q27. What are window functions like rolling, shift and rank in Pandas?

Answer: rolling computes moving-window stats (moving average); shift moves values up or down to compare with the previous row; rank assigns ranks. They mirror SQL window functions.

```
df['ma7'] = df['sales'].rolling(7).mean()
df['prev'] = df['sales'].shift(1)
df['growth'] = df['sales'].pct_change()
df['rank'] = df['sales'].rank(ascending=False)
```

Q28. merge: how do you find rows that did not match?

Answer: Use a left merge with indicator=True and filter the _merge column. It is the Pandas version of a SQL anti-join.

```
m = a.merge(b, on='id', how='left', indicator=True)
unmatched = m[m['_merge'] == 'left_only']
```

Part 3 - Matplotlib

Matplotlib is the base plotting library. Figure = the whole canvas. Axes = one chart inside it. Always label what you plot.

Quick Notes

- Use the object-oriented style: fig, ax = plt.subplots(). It scales better than plt.plot.
- Every chart needs: title, axis labels with units, legend (if more than one series), readable tick labels.
- Chart choice: line = trend over time, bar = compare categories, hist = distribution, scatter = relationship, pie = only for few parts of a whole.

Q29. Figure vs Axes?

Answer: Figure is the whole window or canvas; Axes is one plot area inside it with its own x/y axis. One figure can hold many axes (subplots).

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot([1, 2, 3], [10, 20, 15])
ax.set_title('Sales'); ax.set_xlabel('Month'); ax.set_ylabel('Revenue')
plt.show()
```

Q30. plt.plot vs the object-oriented ax.plot?

Answer: plt.plot uses the implicit current axes (quick for one chart). ax.plot targets a specific axes, which is clearer and needed when there are several subplots or when the code is reused.

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4))
ax1.bar(['A', 'B'], [5, 8]); ax2.hist(data, bins=10)
```

Q31. Which chart do you pick for which data?

Answer: Line for trends over time, bar for comparing categories, histogram for the distribution of one numeric variable, scatter for the relationship between two numeric variables, boxplot to compare distributions and show outliers, pie only for a few parts of a whole.

Q32. How do you save a figure in good quality?

Answer: Use fig.savefig with dpi=300 and bbox_inches="tight" so labels are not cut off. Call it before plt.show().

```
fig.savefig('sales.png', dpi=300, bbox_inches='tight')
```

Q33. How do you make several subplots without overlap?

Answer: Create them with plt.subplots(rows, cols) and call fig.tight_layout() (or constrained_layout=True) so titles and labels do not collide.

```
fig, axes = plt.subplots(2, 2, figsize=(10, 8), constrained_layout=True)
axes[0, 0].plot(x, y)
```

Q34. How do you customise a chart for a presentation?

Answer: Set colors and linewidth, add annotations for key points, remove top and right spines, rotate long tick labels, and use a consistent style such as plt.style.use("seaborn-v0_8").

```
ax.spines[['top', 'right']].set_visible(False)
ax.annotate('Peak', xy=(5, 90), xytext=(6, 95), arrowprops=dict(arrowstyle='->'))
plt.xticks(rotation=45)
```

Q35. hist vs bar chart?

Answer: A histogram bins one continuous numeric variable and shows its distribution (bars touch). A bar chart compares separate categories (bars have gaps).

Part 4 - Seaborn

Seaborn sits on top of Matplotlib. It works directly with DataFrames, gives good-looking statistical charts with little code, and handles grouping with the hue argument.

Quick Notes

- Pass the DataFrame with data=df and column names as strings: sns.barplot(data=df, x="city", y="sales").
- hue splits by a category into colors. Figure-level functions (relplot, catplot, displot) can create facets with col= / row=.
- sns.set_theme() applies a clean default style. Seaborn returns a Matplotlib Axes, so you can still customise with ax.set_title() etc.

Q36. Matplotlib vs Seaborn - when to use which?

Answer: Matplotlib gives full low-level control. Seaborn is a higher-level layer for statistical charts with better defaults, direct DataFrame support and automatic aggregation. In practice you use Seaborn to draw and Matplotlib to fine-tune.

Q37. Which Seaborn plots are most used in EDA?

Answer: histplot/kdeplot for distributions, boxplot/violinplot for spread and outliers, scatterplot for relationships, heatmap for correlation matrices, countplot for category counts, pairplot for many variables at once.

```
sns.histplot(data=df, x='sales', kde=True)
sns.boxplot(data=df, x='city', y='sales')
sns.pairplot(df[['sales', 'qty', 'profit']])
```

Q38. How do you draw a correlation heatmap?

Answer: Compute df.corr(numeric_only=True) and pass it to sns.heatmap with annot=True. Values near +1 or -1 mean strong correlation; near 0 means weak.

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
```

Q39. What does hue do?

Answer: hue maps a categorical column to color so one chart shows several groups together. It adds a legend automatically.

```
sns.scatterplot(data=df, x='qty', y='sales', hue='city')
```

Q40. barplot vs countplot?

Answer: countplot counts rows per category for you (only needs x). barplot plots an aggregate of a numeric column per category (mean by default, with a confidence interval).

```
sns.countplot(data=df, x='city')
sns.barplot(data=df, x='city', y='sales', estimator='sum', errorbar=None)
```

Q41. How do you show a distribution comparison across groups?

Answer: Use boxplot or violinplot (x = group, y = numeric). A violin also shows the density shape. Use displot with hue or col to compare histograms.

```
sns.violinplot(data=df, x='segment', y='profit')
sns.displot(data=df, x='profit', hue='segment', kind='kde')
```

Part 5 - Scenario and Rapid-fire Questions

Short questions interviewers ask to test how you actually work with data.

Q42. You get a new CSV. What are your first 5 steps in Pandas?

Answer: 1) read_csv and check shape. 2) info() for dtypes and nulls. 3) describe() for ranges. 4) check duplicates and missing values. 5) value_counts on key categorical columns. Only then clean and analyse.

```
df = pd.read_csv('data.csv')
df.shape; df.info(); df.describe()
df.isna().sum(); df.duplicated().sum()
```

Q43. Explain your Netflix Data Analysis project using these libraries.

Answer: I loaded the titles dataset with Pandas, cleaned nulls and duplicates, split multi-value columns (genres, countries), converted dates with to_datetime, used groupby and value_counts for trends, and presented the findings with Matplotlib and Seaborn charts such as release-year trends and content type split.

Q44. Total sales per city, top 3 cities - write it in Pandas.

Answer: Group by city, sum sales, sort descending and take the top 3 with nlargest.

```
top3 = df.groupby('city')['sales'].sum().nlargest(3)
print(top3)
```

Q45. Which is faster for 1 million rows: a for-loop, apply or vectorized operation? Why?

Answer: Vectorized is fastest (compiled C over whole arrays), then apply (Python function per row), then a for-loop (slowest, interpreter overhead each iteration). Choose vectorized operations whenever possible.

Q46. How would you explain a chart choice to a non-technical stakeholder?

Answer: State the question first (trend? comparison? distribution?), pick the chart that answers it directly, keep one message per chart, label it clearly, and highlight the key number or point with an annotation.

Last-minute Cheat Sheet

Task	Code
Preview data	`df.head(); df.info(); df.describe()`
Nulls / duplicates	`df.isna().sum(); df.duplicated().sum()`
Fill / drop nulls	`df['c'].fillna(x); df.dropna(subset=['c'])`
Filter rows	`df[(df.a > 5) & (df.b == 'x')]`
Group and aggregate	`df.groupby('k').agg(total=('v','sum'))`
Join tables	`pd.merge(a, b, on='id', how='left')`
Dates	`pd.to_datetime(df['d']); df['d'].dt.month`
Rolling average	`df['v'].rolling(7).mean()`
Pivot	`pd.pivot_table(df, index='a', columns='b', values='v')`
NumPy mask / where	`a[a > 10]; np.where(a > 10, 1, 0)`
Line / bar / hist	`ax.plot(x, y); ax.bar(x, y); ax.hist(v)`
Heatmap	`sns.heatmap(df.corr(numeric_only=True), annot=True)`